

QR Code Tracking (No-PTZ)

1. Experiment Objective

The camera tracks a QR code, and the car rotates to keep the QR code at the center of the frame. It supports Tracking mode (rotation only), Follow mode (rotation + distance keeping), and Control mode (decoded content drives actions).

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
QR Code Tracking	Detects the position and boundaries of a QR code, computes its center coordinates, and drives the car to track it
pyzbar Decoding	Decodes QR code images based on the open-source pyzbar library to extract the encoded content
Stable-frame Gating	Confirms tracking only after the same target is detected for N consecutive frames, avoiding false detections and transient interference
Lost-frame Timeout	After the target is lost for longer than the set time, it is considered lost and the car stops moving
PD Control	Proportional-derivative controller that computes the body angular velocity based on the QR code center deviation and its rate of change
QoS	best_effort (image input + cmd_vel downlink), depth=1

2. Technical Architecture

<pre>/camera/image_raw (BEST_EFFORT) → qr_tracker_node └─ Image processing timer: image conversion → flip → pyzbar/OpenCV QR detection → select the largest code └─ Body control timer (10Hz): pixel error → PD angular velocity + QR code size distance following → /cmd_vel (BEST_EFFORT)</pre>

3. Core Topic Quick Reference

Firmware Uplink

Topic	Message Type	QoS	Description
/camera/image_raw	sensor_msgs/Image	best_effort	ESP32 camera image

Firmware Downlink

Topic	Message Type	QoS	Field	Range	Direction
/cmd_vel	geometry_msgs/Twist	best_effort + keep_last(1)	linear.x	-0.18~0.18 m/s	$\pm$ forward, $\mp$ backward
			angular.z	-0.60~0.60 rad/s	$\pm$ turn left, $\mp$ turn right

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	✗
No-PTZ version	☑
Required peripheral	ESP32 camera (bracket-mounted), and a QR code that is printed or displayed on a phone

2. Start the Agent + Camera + Joint Model (Terminal 1)

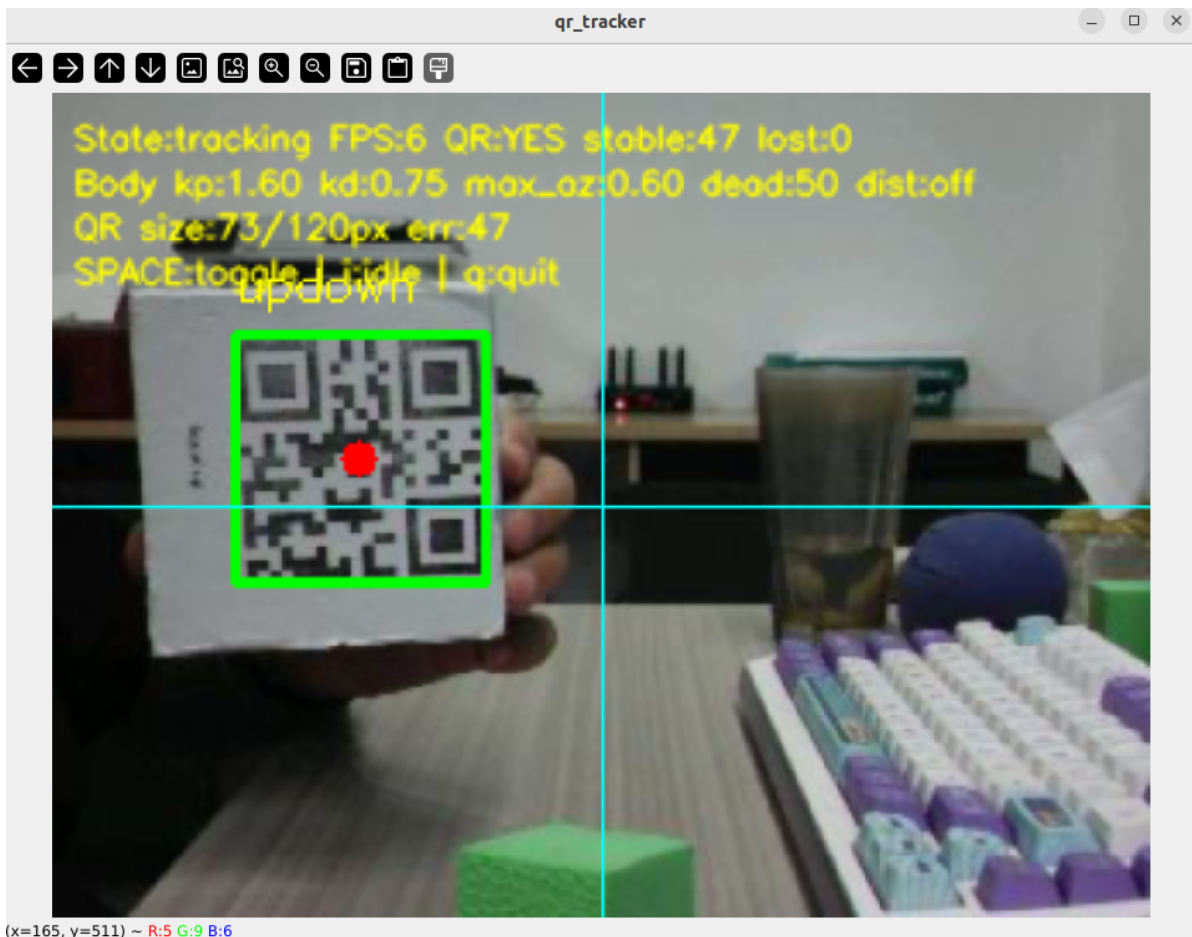
```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start QR Code Tracking (Terminal 2)

Tracking mode (rotation tracking only):

```
ros2 launch microros_v2_no_ptz_visual_control_pkg qr_tracker.launch.py
```



4. Operation Instructions

Key	Function
Space	Toggle tracking/standby state
q / Q / ESC	Exit the program

4. Experiment Phenomenon

- When the QR code moves left or right in the frame, the car automatically rotates to track it and keeps it at the center of the frame
- The terminal outputs the decoded QR code content in real time
- Stable-frame gating: tracking is confirmed only after 3 consecutive detections
- After the lost-marker timeout, the car stops moving
- Follow mode: the car backs away when the QR code moves closer and moves forward when it moves away

5. Code Explanation

Source path:

- Launch (track):
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/qr_tracker.launch.py`
- Launch (follow):
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/qr_follow.1`

aunch.py

- Node source:

```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/qr_tracker_node.py
```

1. Core Node Analysis

`qr_tracker_node` shares the pyzbar/OpenCV dual-backend QR detection logic with `qr_control_node`, but uses a tracking architecture (PD rotation + distance following) instead of control-command mapping.

Constructor — declare parameters, create the dual-backend detector and publishers/subscribers:

```
# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visua
l_control_pkg/qr_tracker_node.py

def __init__(self) -> None:
    super().__init__(DEFAULT_NODE_NAME)
    self._tgt_lock = threading.Lock()
    self._state = STATE_IDLE
    self._has_target = False
    self._lost_frames = 0
    self._stable_count = 0          # Stable-frame gating counter
    self._target_x: Optional[float] = None
    self._target_y: Optional[float] = None
    self._target_size: Optional[float] = None # QR code bounding rectangle size
    self._payload: str = ""        # Decoded QR code content

    self._declare_params()
    self._load_params()

    # pyzbar/OpenCV dual backend
    self._opencv_qr = cv.QRCodeDetector()

    best_effort = QoSProfile(
        history=QoSHistoryPolicy.KEEP_LAST, depth=1,
        reliability=QoSReliabilityPolicy.BEST_EFFORT,
        durability=QoSDurabilityPolicy.VOLATILE,
    )
    self._img_sub = self.create_subscription(
        Image, self._img_topic, self._on_image, best_effort)
    self._cmd_pub = self.create_publisher(Twist, self._cmd_topic, best_effort)

    self._img_timer = self.create_timer(
        1.0 / max(float(self._img_rate), MINIMUM_TIMER_RATE_HZ),
    self._process_image)
    self._body_timer = self.create_timer(
        1.0 / max(float(self._body_rate), MINIMUM_TIMER_RATE_HZ),
    self._body_tick)
```

Body control callback — stable-frame gating + PD tracking:

```

# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visua
l_control_pkg/qr_tracker_node.py

def _body_tick(self) -> None:
    """Body control timer (10Hz)."""
    if self._state != STATE_TRACKING or self._shutdown:
        self._pub_zero()
        return

    with self._tgt_lock:
        has = self._has_target
        tx = self._target_x
        ts = self._target_size

    # Stable-frame gating: output nothing when consecutive frames are below the
    threshold
    if self._stable_count < self._follow_stable_frames:
        self._pub_zero()
        return

    if not has or tx is None:
        self._pub_zero()
        return

    # Horizontal pixel error → PD angular velocity
    cx = self._img_w * 0.5
    err_x = tx - cx
    if abs(err_x) <= self._deadband_x:
        az = 0.0
    else:
        now = time.perf_counter()
        dt = now - self._prev_t if self._prev_t > 0 else 1.0 / self._body_rate
        dt = max(dt, 1e-3)
        d_err = (err_x - self._prev_err) / dt
        self._prev_err = err_x; self._prev_t = now
        az = self._angular_kp * err_x + self._angular_kd * d_err
        az = clamp(az, -self._max_az, self._max_az)
        if self._reverse_body:
            az = -az

    cmd = Twist()
    cmd.angular.z = az

    # Distance following: QR code size deviation → PD linear velocity
    if self._dist_follow and ts is not None:
        err_s = self._target_qr_size - ts
        if abs(err_s) > self._dist_tol_qr:
            lx = self._dist_kp * err_s
            lx = clamp(lx, -self._max_lx, self._max_lx)
            if self._reverse_linear:
                lx = -lx
            cmd.linear.x = lx

    self._cmd_pub.publish(cmd)

```

Key logic: Stable-frame gating (`_follow_stable_frames=3`) requires the QR code to be detected for 3 consecutive frames before tracking is confirmed, preventing occasional false detections. A decoding failure (e.g., a blurry QR code) does not affect tracking; it only affects the payload display.

2. Key Parameters

Parameter definition file:

- Launch:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/qr_tracker.launch.py`
- Node:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/qr_tracker_node.py`

Parameter	Type	Default	Description
<code>qr_detector_backend</code>	str	<code>pyzbar</code>	QR decoding backend (pyzbar / opencv)
<code>qr_select_strategy</code>	str	<code>largest</code>	Multi-QR-code selection strategy
<code>follow_stable_frames</code>	int	<code>3</code>	Stable-frame gating: confirm tracking only after N consecutive detections
<code>lost_timeout</code>	float	<code>0.5 s</code>	Lost-marker timeout
<code>deadband_x_pixel</code>	float	<code>50.0 px</code>	Horizontal deadband
<code>angular_kp</code>	float	<code>1.6</code>	Body angle PD proportional coefficient
<code>angular_kd</code>	float	<code>0.75</code>	Body angle PD derivative coefficient
<code>max_angular_z</code>	float	<code>0.6 rad/s</code>	Maximum body steering speed
<code>enable_distance_follow</code>	bool	<code>false</code>	Whether to enable distance following
<code>target_qr_size_px</code>	float	<code>120.0 px</code>	Target QR code pixel size for distance following
<code>distance_tolerance_qr_size_px</code>	float	<code>20.0 px</code>	Distance-following deadband
<code>distance_kp</code>	float	<code>0.05</code>	Distance PD proportional coefficient
<code>max_linear_x</code>	float	<code>0.18 m/s</code>	Maximum forward/backward speed

3. Node Description

Node	Package	Description
<code>qr_tracker_node</code>	<code>microros_v2_no_ptz_visual_control_pkg</code>	QR code tracking with stable-frame gating + PD body control

6. Common Problems

Problem	Cause	Solution
Frequent marker loss	QR code too small or moving too fast	Increase the QR code size and slow down the movement
Decoding failure	QR code blurry, reflective, or distorted	Ensure the QR code is clear, flat, and evenly lit
Tracking does not start	Stable-frame gating threshold not met	Keep the QR code still for 3 frames or more